

Class Structure Approach

1. Grid with Sprite Actor

1.1 Grid Actor Class acts as a manager which handles most of the game logic.

It holds a 2D array of pointers of type Tile. It will spawn a 2D array of tile actors and store information in it.

Variables:

- Rows: Number of rows in the grid.
- Columns: Number of columns in the grid.
- TileArray: 2D array of pointers to Tile instances.
- GameState: Enum (e.g., Idle, Swapping, Animating, GameOver).
- Score: Integer to keep track of the player's score.

Functions:

- GenerateGrid(): Initializes and spawns the grid of tiles.
- OnTileClickReceive(tileIndex): Handles tile click events.
- CheckForMatches(): Evaluates the grid for matching tiles after a move(in swap functionality)
- UpdateScore(points): Updates the player's score.
- RefreshGrid(): Resets the grid after matches are cleared.
- HandleSwap(*tileA, *tileB): Manages tile swapping logic.

1.2 Tile actor denotes a physical fruit/gem on screen.

This tile class is extended by subclasses to implement different fruit/gem subclasses.

Grid => Tile => Child Classes => Specific Classes
Class Fruit :: ATile, Class Gem:: ATile

Variables:

SpriteComponent: Represents the visual appearance of the tile.

SpriteIndex: Index of the sprite in the texture atlas.

TileType: Enum (e.g., Fruit, Gem).

Position: 2D vector indicating the tile's position in the grid.

Functions:

OnClick(): Triggered when the tile is clicked.

Animate(): Triggers the appropriate animation based on the type.

1.3 Subclasses

Fruit Class: Extends Tile

Contains specific properties or methods related to fruits.

Example properties: FruitType, FruitScoreAdd.

Gem Class: Extends Tile

Contains specific properties or methods related to gems.

Example properties: GemType, GemPowerUp.

1.3 On Click will be applied on paper sprite which then will share its information with the grid for further manipulations as in

Fruits will be calling for Clustering logic

Gems will be calling for Clearance logic

1.4 Animation for both the subclasses will be a variable,

Animation Variables:

ClusteringAnimation: Animation sequence for fruit clustering.

ClearanceAnimation: Animation sequence for gem clearance.

Functions:

ApplyFruitAnimation(): Triggers the animation for clustered fruits.

ApplyGemAnimation(): Triggers the animation for cleared gems.

1.5 SWAP Integration

Grid would handle multiple clicks

OnTileClickReceive(tileIndex)

Functions:

ifSwapValid(*tileA, *tileB)

performSwap()

CheckForMatches()